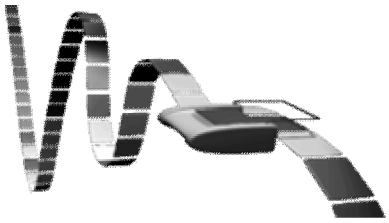
CHAPTER - 4

Turing Machine and Undecidability

4.1 Introduction to Turing Machine



Turing machine provides an ideal theoretical model (as it has infinite memory on tape) of a computer that accepts type-0 language. It is used for computing functions & turns out to be a mathematical model of partial recursive functions. Turing machines are also used for determining the un-decidability of certain language and measuring the space & time complexity.

A Turing machine is a mathematical model of a general computing machine. It is a theoretical device that manipulates symbols contained on a strip of tape. Turing machines are not intended as a practical computing technology, but rather as a thought experiment representing a computing machine. It is believed that if a problem can be solved by an algorithm, there exist a Turing machine that solves the problem. The Turing machine is the most commonly used model in complexity theory to define different complexity class problems such as class P-problems, class NP-problems, and class EXPSPACE-problems etc.

Many types of Turing machines are used to define complexity classes, such as deterministic Turing machines, probabilistic Turing machines, non-deterministic Turing machines, quantum Turing machines, symmetric Turing machines and alternating Turing machines. They are all equally powerful in principle, but when resources (such as time or space) are bounded, some of these may be more powerful than others.

Definition

Turing machine can be thought as a finite state automata connected to an R/W (read/write) head. It has one tape which is divided into a number of squares or cells. The block diagram of the basic model for Turing Machine is given as shown in figure:

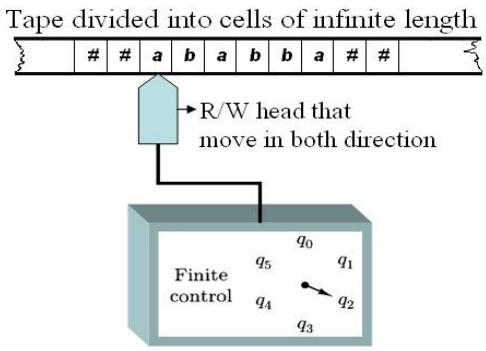


Fig. B.D. of a Turing Machine

- The machine consists of a finite control that can be in any of a finite set of states. The tape is divided into numbers of cells each can holds any one of a finite number of symbols.
- Initially, the input, which is a finite length string of symbols chosen from the input alphabet, is placed on the tape. All other tape cells, extending infinitely to the left & right, initially holds the special symbols called the ***blank*** (#). The blank is a tape symbol but not the input symbol.
- There is a tape head that is always positioned at one of the tape cell where the Turing Machine is said to be scanning that cell. Initially, the tape head is at the leftmost cell that holds the input string.

Mathematically, the Turing Machine, $M = (Q, \Sigma, \Gamma, \delta, q_0, \#, F)$ can be described by the 7-touple, where

Q = Finite non-empty set of states

Σ = Finite non-empty set of input alphabet or symbols

Γ = Complete set of tape symbols (Σ + #*)

q_0 = Initial states or start states ($q0 \in Q$)

= Blank symbol (# $\in \Gamma$)

F = Subset of Q (i.e. $F \subseteq Q$) consisting of final state (i.e. one or more accepting states).

δ = Transition function, which define the move of Turing machine of the form δ(q, x) → (p, y, D).

Here,

q = Current state in Q
x = Tape symbol
p = Next state in Q defined after the current state
y = Symbol in Γ written in the cell being scanned, replacing whatever symbol was there
D = Direction either Left (L) or Right (R) or No-move (N), in which the head move.

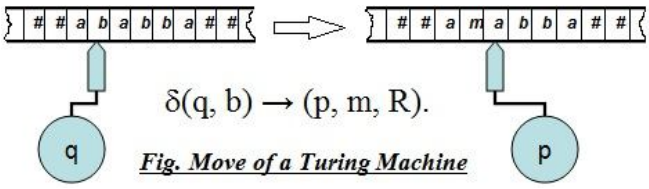


Fig. Move of a Turing Machine

4.2 Representation of Turing Machine

A. Representation of TM by Instantaneous Descriptions (ID)

An ID of an Turing Machine 'M' is a string ***α q b β,*** where 'q' is the present state of M, the entire string is split as ***αβ+b,*** the symbol 'b' is the current symbol under the R/W head, the 'α' represent the left sub-string of the input string & 'β' represent the right sub-string of the input string.

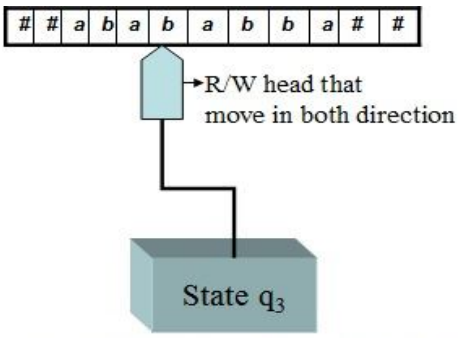


Fig. A Snapshot of Turing Machine

The ID shows the action for one *instant* of time or 'Snapshots' of a Turing Machine. Here, we use the term instant because it explain the state information, direction of move & write facility, thus we cannot use the same terminology for the Finite Automata or PDA.

Here, the present symbol under R/W head is ***'b'*** & the present state is ***'q_3'***. Thus, ***'b'*** must be written to the right of ***'q_3'***. The non-blank symbol to the left of 'b' is ***'aba'*** that must be written to the left of ***'q_3'***. The sequence of non-blank symbols to the right of 'b' is ***'abba'***. Hence, the ID can be shown in the following figure. The $\delta(q_3, b)$ induces a change in ID of the TM. We call this change in ID a ***move***.

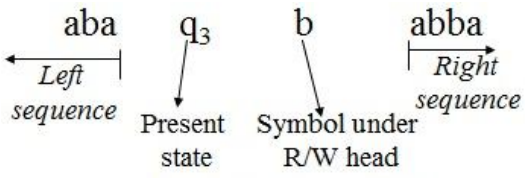


Fig. Representation of ID

- Suppose, $\delta(q, x_i) = (p, y, L)$ is a move of TM, where the input string can be represented as $x_1, x_2 \ldots x_n$. Here, the present symbol under R/W head is 'x_i'.
  Thus,

  ID before processing

  $x_1, x_2 \ldots x_{i-1}$ **q** $x_i, x_{i+1} \ldots x_n$

  ID after processing

  $x_1, x_2 \ldots x_{i-2},$ **p** x_{i-1} y, $x_{i+1} \ldots x_n$

  This change of ID is represented by

  $x_1, x_2 \ldots x_{i-1}$ **q** $x_i, x_{i+1} \ldots x_n \rightarrow x_1, x_2 \ldots x_{i-2},$ **p** x_{i-1} y, $x_{i+1} \ldots x_n$

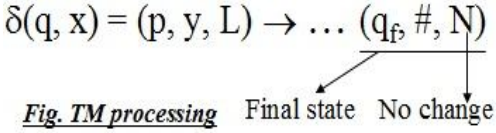


Fig. TM processing

- If $\delta(q, x_i) = (p, y, R)$ then the change of ID is represented by

  $x_1, x_2 \ldots x_{i-1}$ **q** $x_i, x_{i+1} \ldots x_n \rightarrow x_1, x_2 \ldots x_{i-2}, x_{i-1}$ y, **p** $x_{i+1} \ldots x_n$

B. Representation of TM by Transition Table

We give the definition of δ in the form of a table called transition table. If δ(q, x) = (p, y, β), it means that 'x' is the current symbol under the R/W head that will replace by 'y' after transition, β gives the movement of the head (L or R or N) and 'p' denotes the new state into which the Turing Machine enters. An example can be seen in figure.

Q/Σ	0	1	#
→q_0	(q_0, 0, R)	(q_1, 0, R)	-
q_1	-	(q_2, 1, R)	(q_0, #, L)
(q_2)	(q_1, 1, R)	(q_2, 0, L)	(q_0, 1, R)

Where, Σ= Tape symbol
Q = Present state

Fig. Transition Table of a TM

C. Representation of TM by Transition Diagram

A transition diagram of a TM is a graphical representation consisting of a finite number of nodes and directed labeled arcs between the nodes. Each node represents a state of the TM and a label on arc from one state to another state represents the information about the required input for transition. The label on arc is generally written as 0 / (1, β) where β gives the movement of the head in L or R or N.

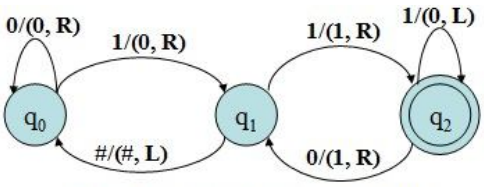


Fig. Transition Diagram for TM

4.3 Design of Turing machine

The Turing machines are designed to play following different roles:

- TM can use as language accepter similar to the role played by the Finite Automata & PDAs.
- TM can be used as computing function. In this role, a TM represents a particular function. The initial input is treated as representing an argument of the function and the final symbol on the tape when the TM enters the halt state treated as representative of the value obtained by an application of the function to the argument represented by the initial string.
- TM can be used as enumerator of the strings of a language that outputs the string of a language, one at a time, in some systematic order that is on a list.
- TM can able to differentiate any language whether it is decidable or un-decidable.
- TM can able to differentiate any problem whether it is in class-P or class-NP.

Basic guidelines for designing a Turing machine

a. The fundamental objective in scanning a symbol by the R/W head is to 'know' what to do in the future. The machine must remember the past symbol scanned. The Turing machine can remember this by going to the next unique state.
b. The number of state must be minimized. This can be achieved by changing the state only when there is a change in the written symbol or when there is a change in moment of the R/W head.

A. Turing machine as Language acceptor

A TM works as a language acceptor for a language if it is able to tell us whether a string 'w' belongs to the language L or not.

Let us consider the Turing machine M = (Q, Σ, Γ, δ, q_0, #, F). A string 'w' in Σ* is said to be accepted by M if $q_0 w \text{-}^* \rightarrow \alpha_1 p \alpha_2$ for some $p \in F$ and $\alpha_1, \alpha_2 \in \Gamma^*$. Here, 'p' is any final state determined after reading the whole input string. The Turing machine 'M' does

not accept 'w' if the machine 'M' either halts in a non-accepting state or reach to any non-accepting state after read the whole input string.

There are two ways for acceptance of a string by Turing machine:

a. Acceptance by Final State

Here, the TM must reach to the any one final state after processing the whole input string.

For example

Let a Turing Machine, M = (Q, Σ, Γ, δ, q_0, #, F) can be described by the 7-touple, where

Q = Set of states = {q_0, q_1}
Σ = Set of input alphabet or symbols = {0, 1}
Γ = Set of tape symbols (Σ + #*) = {0, 1, #}
q_0 = Initial states or start states ($q_0 \in Q$)
= Blank symbol (# ∈ Γ)
q_1 = Final state
δ = Transition function, which define by the given transition diagram.

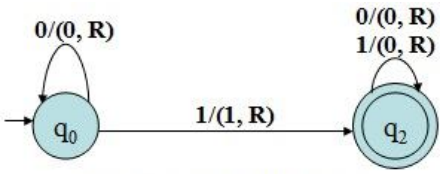


Fig. Transition Diagram

Now, check whether M accept the given string (w) = 00011 or not.

Solution

Here, transition function can be defined as:

$\delta(q_0, 0) \rightarrow (q_0, 0, R)$
$\delta(q_0, 1) \rightarrow (q_1, 1, R)$
$\delta(q_1, 0) \rightarrow (q_1, 0, R)$
$\delta(q_1, 1) \rightarrow (q_1, 1, R)$

Now, the given string (w) = 00011 can be process as:

$q_0 00011 \rightarrow 0q_0 0011 \rightarrow 00q_0 011 \rightarrow 000q_0 11 \rightarrow 0001q_1 1 \rightarrow 00011q_1\# \text{ or } \rightarrow 00011q_1$

Since the TM reach to the final state after processing all input string so we say that it is accepted by TM.

b. Acceptance by Halting State

Here, we first declare the list of all the halt states, and then we examine that the resulting state (beside any final state) after processing all the input symbols is whether the defined halting state or not for accepting state. Thus, we can conclude that if the TM goes into the halting state before reading the whole input symbol & further move is not defined then we say that the TM cannot accept the given string. This can be represented as: $(q_0, \#) \rightarrow (h, \#, N)$, where 'h' is any one defined halting state & 'N' is no direction for move.

Example-1: Design a TM that erases all non-blank symbols on the tape.

Solution

Let the required Turing machine, M = (Q, Σ, Γ, δ, q_0, #, F), where
Q = Set of states = {q_0, h}, where h = halting state
Σ = Set of input alphabet or symbols = {a, b}
Γ = Set of tape symbols (Σ + #*) = {a, b, #}
q_0 = Initial states or start states ($q_0 \in Q$)

= Blank symbol (# ∈ Γ)
δ = Transition function, which defines as:

$\delta(q_0, a) \rightarrow (q_0, \#, R)$
$\delta (q_0, b) \rightarrow (q_1, \#, R)$
$\delta (q_0, \#) \rightarrow (h, \#, N)$
$\delta (h, \#) \rightarrow$ Accepting state

Present state	***Tape symbol***		
	a	***b***	***#***
$\rightarrow q_0$	$(q_0, \#, R)$	$(q_0, \#, R)$	(h, #, N)
h			Accepting state

Now, let any string (w) = abbbaa, which can be process as:
q_0 abbbaa $\rightarrow$ $\#q_0$bbbaa $\rightarrow$ $\#\#0q_0$bbaa $\rightarrow$ $\#\#\#q_0$baa $\rightarrow$ $\#\#\#\#q_1$aa $\rightarrow$ $\#\#\#\#\#q_1$a $\rightarrow$ $\#\#\#\#\#\#q_1\# \rightarrow$ ######h

Since the TM reach to the defined halting state after processing all input string so we say that it is accepted by TM. Hence, the defined TM is capable for handling our problem.

Present state	***Tape symbol***	
	a	***b***
$\rightarrow q_0$	$(q_1, \#, R)$	$(q_1, \#, R)$
$*q_0$	$(q_1, \#, R)$	$(q_1, \#, R)$

Note: This problem also can be treated by defining the final state rather than the halting state as:

Example-2: Design a TM that recognizes the language of all the string of even length over the alphabet {a, b}.

Solution

Let the required Turing machine, M = (Q, Σ, Γ, δ, q_0, #, F), where
Q = Set of states = {q_0, q_1, h}, where h = halting state
Σ = Set of input alphabet or symbols = {a, b}
Γ = Set of tape symbols (Σ + #*) = {a, b, #}
q_0 = Initial states or start states ($q_0 \in Q$)
= Blank symbol (# ∈ Γ)
δ = Transition function, which defines as:

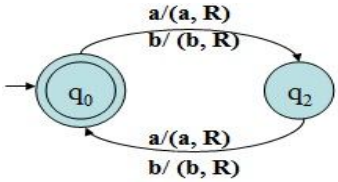


Fig. Transition Diagram

$\delta(q_0, a) \rightarrow (q_1, a, R)$
$\delta (q_0, b) \rightarrow (q_1, b, R)$
$\delta(q_1, a) \rightarrow (q_0, a, R)$
$\delta (q_1, b) \rightarrow (q_0, b, R)$
$\delta (q_0, \#) \rightarrow (h, \#, N)$
$\delta (h, \#) \rightarrow$ Accepting state

Present state	***Tape symbol***		
	a	***b***	***#***
$\rightarrow q_0$	(q_1, a, R)	(q_1, b, R)	(h, #, N)
q_1	(q_0, a, R)	(q_0, b, R)	
h			Accepting state

Now, let any string (w) = abbbaa, which can be process as:
q_0 abbbaa $\rightarrow$ aq_1bbbaa $\rightarrow$ abq_0bbaa $\rightarrow$ $abbq_1$baa $\rightarrow$ $abbbq_0$aa $\rightarrow$ $abbbaq_1$a $\rightarrow$ $abbbaaq_0\# \rightarrow$ ######h

Since the TM reach to the defined halting state after processing all input string so we say that it is accepted by TM. Hence, the defined TM is capable for handling our problem.

B. Turing machine for computing function

The Turing machine can be used to compute some functions that are called the *Turing computable function.*

Let Σ_0, Σ_1 be any alphabets not containing the blank symbol #. Also, let 'f' be a function from Σ_0^* to Σ_1^*. A Turing machine 'M' is said to compute 'f' if Σ_0, $\Sigma_1 \subseteq \Sigma$ and for any string $w \in \Sigma_0^*$ if f (w) = u then

$$(S, \# w \underline{\#}) \xrightarrow[M]{*} (h, \# u \underline{\#})$$

Where, h is the halting state.

If some such Turing machine 'M' exists, then the function 'f' is said to be a Turing computable function.

Note:

a. Here, the TM must be halt when it find the resulting functional value during processing from the start state's'.
b. Σ is the input symbol for the defined TM.
c. $\subseteq$ is proper subset which indicates subset or same set
d. U is any resulting value of function 'f'
e. Function 'f' from Σ_0^* to Σ_1^* shows that the both set of input symbols are fall on the same domain i.e. real number to real number etc.
f. The # shows the current position of the reading head.

Example-1: Design a Turing machine which compute the function f (n) = n+1, for $n \in N$ (natural number).

Solution

Here, we design a Turing machine 'M' which computes any function 'f' by writing an 'I' in the tape square in which its head is initially located, moving its head one square right and halting.

Formally, let the required Turing machine, M = (Q, Σ, Γ, δ, q_0, #, F), where
Q = Set of states = $\{q_0, h\}$, where h = halting state
Σ = Set of input alphabet or symbols = {I}
Γ = Set of tape symbols (Σ + #*) = {I, #}
q_0 = Initial states or start states ($q_0 \in Q$)
= Blank symbol (# $\in \Gamma$)
δ = Transition function, which defines as:

$\delta(q_0, \#) \rightarrow (h, I, R)$
OR
$\delta(q_0, \#) \rightarrow (q_0, I, N)$
$\delta(q_0, I) \rightarrow (h, \#, R)$
$\delta(h, \#) \rightarrow$ Accepting state

Present state	***Tape symbol***	
	I	***#***
$\rightarrow q_0$	(h, #, R)	(q_0, I, N)
h		Accepting state

In case of $\delta(q_0, \#) \rightarrow (h, I, R)$

For n=0
- $\delta(q_0, \underline{\#}\#) \rightarrow (h, I\underline{\#})$ i.e. Accepted

For n=1
- $\delta(q_0, \#I\underline{\#}\#) \rightarrow (h, \#II\underline{\#})$ i.e. Accepted

For n=2
- $\delta(q_0, \#II\underline{\#}\#) \rightarrow (h, \#III\underline{\#})$ i.e. Accepted

Hence, in similar manner we can conclude that
For $n=I^n$
- $\delta(q_0, \# I^n\underline{\#}\#) \rightarrow (h, \# I^{n+1}\underline{\#})$ i.e. Accepted

In case second

For n=0
- $\delta(q_0, \underline{\#}\#) \rightarrow (q_0, \underline{I}\#) \rightarrow (h, I\underline{\#})$ i.e. Accepted

For n=1
- $\delta(q_0, I\underline{\#}\#) \rightarrow (q_0, I\underline{I}\#) \rightarrow (h, II\underline{\#})$ i.e. Accepted

For n=2
- $\delta(q_0, II\underline{\#}\#) \rightarrow (q_0, III\#) \rightarrow (h, III\underline{\#})$ i.e. Accepted

Hence, in similar manner we can conclude that
For n=0
- $\delta(q_0, I^n\underline{\#}\#) \rightarrow (q_0, I^nI\#)$ or $(h, I^{n+1}\underline{\#})$ i.e. Accepted

Hence, the Turing machine 'M' computes the given function.

Example-2: Design a Turing machine which compute the function f (n) = n+2, for n∈N (natural number).

Solution

Here, we design a Turing machine 'M' which computes any function 'f' by writing an 'I' in the tape square in which its head is initially located, moving its head one square right and halting.

Formally, let the required Turing machine, M = (Q, Σ, Γ, δ, q_0, #, F), where
Q = Set of states = {q_0, h}, where h = halting state
Σ = Set of input alphabet or symbols = {I}
Γ = Set of tape symbols (Σ + #*) = {I, #}
q_0 = Initial states or start states ($q_0 \in Q$)
= Blank symbol (# ∈ Γ)
δ = Transition function, which defines as:
$\delta(q_0, \#) \rightarrow (q_1, I, R)$
$\delta(q_1, \#) \rightarrow (h, I, R)$
$\delta (h, \#) \rightarrow$ Accepting state

Present state	***Tape symbol***	
	I	***#***
$\rightarrow q_0$	(q_1, I, R)	
q_1	(h, I, R)	
h		Accepting state

Here,

For n=0
- $\delta(q_0, \underline{\#}\#\#) \rightarrow (q_1, I\underline{\#}\#) \rightarrow (h, II\underline{\#})$ i.e. Accepted

For n=1
- $\delta(q_0, I\underline{\#}\#\#) \rightarrow (q_1, II\underline{\#}\#) \rightarrow (h, III\underline{\#})$ i.e. Accepted

For n=2
- $\delta(q_0, II\underline{\#}\#\#\#) \rightarrow (q_1, III\underline{\#}\#) \rightarrow (h, IIII\underline{\#})$ i.e. Accepted

Hence, in similar manner we can conclude that
For n=0
- $\delta(q_0, I^n\underline{\#}\#\#) \rightarrow (q_1, I^nI\underline{\#}\#) \rightarrow (h, I^nII\underline{\#})$ or $(h, I^{n+2}\underline{\#})$ i.e. Accepted

Hence, the Turing machine 'M' computes the given function.

4.4 Deterministic & Non-deterministic Turing machine

Here, we define the deterministic & non-deterministic approach of the TM on the basis of the *Instantaneous Descriptions (ID)*.

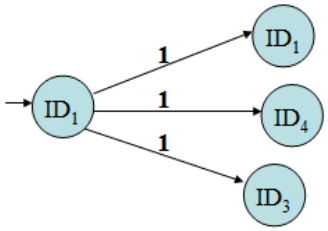
- In case of the Deterministic Turing machine (DTM), the processing of an input symbol results in the transition to a unique ID, so the processing of an input string results in a chain of ID's such as: $\mathbf{ID_1 \rightarrow ID_5 \rightarrow ID_3 \rightarrow \ldots ID_{n-1} \rightarrow ID_n}$

  The transition function $\mathbf{\delta(q_0, a)}$ for the DTM is defined for some elements of set of states (Q) x Set of tape symbols (Γ).

- In case of the Non-deterministic Turing machine (NTM), the processing of an input symbol result in the transition to several IDs.

  The transition function $\mathbf{\delta(q_0, a)}$ for the NTM is defined for some elements of set of states (Q) x Set of tape symbols (Γ) x set of direction of moves {L, R}.

  i.e. $\mathbf{\delta(q_0, a) \rightarrow Q \times \Gamma \times \{L, R\}}$



4.5 Universal Turing Machine

Designing of TM is complex task. We must design a machine that accept two inputs i.e. the input data and a description of algorithm (computation). This is precisely a general purpose computer does. It accepts a data and a program. A general purpose TM is usually called universal TM which is powerful enough to simulate the behavior of any computer, including a TM itself. More precisely, UTM can simulate the behavior of an arbitrary TM over ∑.

Extension of Turing machine

- There may be several tapes instead of only one tape, each having its own dependent need.
- The tape may be allowed to be infinite in both directions.
- There may be more than one head scanning various cells of the tape. Two or more heads simultaneously read the same cell or may attempt to write in the same cell.

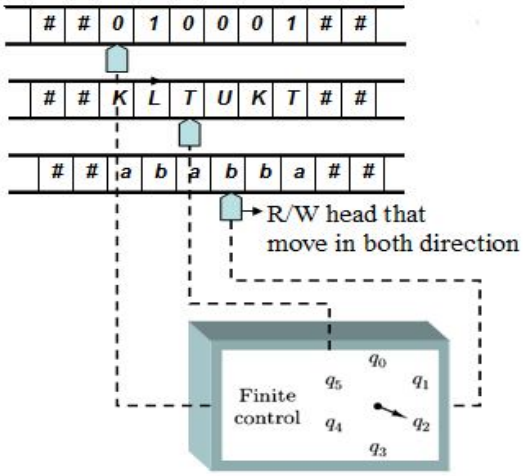


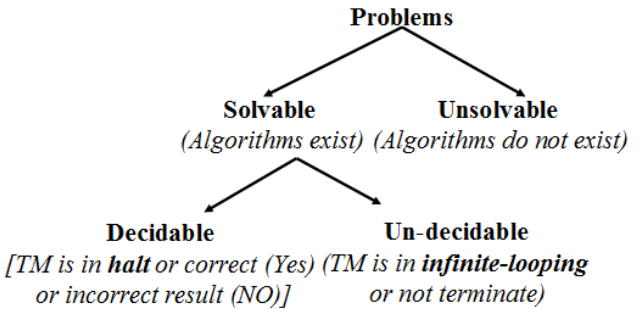
Fig. B.D. of a Turing Machine

4.6 Church's Thesis

Turing machine can be carry out any computation that can be carried out by similar type of automata because these automata seems to capture the essential features of real computing machines, we take TM to be a precise formal equivalent of the intuitive notion of algorithm. Nothing will be considered an algorithm if it cannot be rendered as a TM. The principle that "TMs are formal version of algorithm and that no computational procedure will be considered an algorithm unless it can be represented as a TM" is known as Church's thesis. It is a thesis, not a theorem because it is not a mathematical result.

4.7 Types of Problems

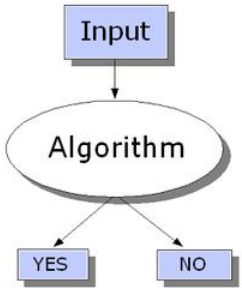
If we compare the language as a problem then it indicates all the solvable problems that include both the decidable & un-decidable problems.



A. Decision problem

In computability theory and computational complexity theory, a **decision problem** is a question in some formal system with a yes-or-no answer, depending on the values of some input parameters. For example, the problem "given two numbers *x* and *y*, does *x* evenly divide *y*?" is a decision problem. The answer can be either 'yes' or 'no', and depends upon the values of *x* and *y*.

Decision problems are closely related to function problems, which can have answers that are more complex than a simple 'yes' or 'no'. A corresponding function problem is "given two numbers *x* and *y*, what is *x* divided by *y*?". They are also related to optimization problems, which are concerned with finding the *best* answer to a particular problem.



A method for solving a decision problem given in the form of an algorithm is called a **decision procedure** for that problem. A decision problem which can be solved by an algorithm, such as this example, is called **decidable**.

B. Function problem

In computational complexity theory, a **function problem** is a computational problem where a single output (of a total function) is expected for every input, but the output is more complex than that of a decision problem, that is, it isn't just YES or NO. Notable examples include the travelling salesman problem, which asks for the route taken by the salesman, and the integer factorization problem, which asks for the list of factors.

Function problems can be sorted into complexity classes in the same way as decision problems. For example FP is the set of function problems which can be solved by a deterministic Turing machine in polynomial time, and FNP is the set of function problems which can be solved by a non-deterministic Turing machine in polynomial time.

C. Search problem

In computational complexity theory and computability theory, a **search problem** is a type of computational problem represented by a binary relation. We need the search problem to select multiple solution from the given large data items but in case of function problem we only select single result of data.

For instance, such problems occurs very frequently in graph theory, where searching graphs for structures such as particular matching, cliques, independent set, etc. are subjects of interest.

4.8 Recursive Language

A language 'L' is recursive language if there exists some Turing machine 'M' that satisfy the following two conditions:

- If $w \in L$, then M accept 'w' *(i.e. reaches on halting state on processing w)* and halts.
- If $w \notin L$, then M eventually halts, without reaching an accepting state.

A TM of this type corresponds to the notation of an algorithm, a well defined sequence of stapes that always finishes and produces an answer. If we think of the language 'L' as a problem, then the problem 'L' is called **decidable** *(if it is a recursive language)* and **un-decidable** *(if it is not a recursive language).*

Recursively Enumerable Language

The recursively enumerable language indicates the set of language that are accepted by using a Turing machine. The terms recursively enumerable means that a function could list all the member of a language in some order.

4.9 Recursive function Theory

A. Partial function

A partial function **f** from X to Y is a rule which assigns to every element of X almost one element of Y. For example, if R denotes the set of all real numbers, then $f(r) = +\sqrt{r}$ is a partial function since f(r) is not defined as a real number when r is negative.

B. Total function

A total function **f** from X to Y is a rule which assigns to every element of X, a unique element of Y. For example, if R denotes the set of all real numbers, then f(r) = 2r is a total function since f(r) is defined for both positive and negative.

C. Primitive Recursive function

The set of primitive function is obtained by three types of initial functions and three structuring rules for constructing more complex function from already constructed functions.

- The three type of initial functions are:
  - **Zero function** defined by Z(x) = 0 e.g. Z(7) = 0
  - **Successor function** defined by S(x) = x + 1, e.g. S(7) = 5
  - **Projection function** defined by $U_i^n(x_1, \ldots x_n) = x$

- The three type of structuring rules are:
  - **Composition**: For example if f_1, f_2 and f_3 are partial functions of two variables and g is a partial function of three variable, then the composition of g with f_1, f_2 and f_3 is given by: g $\{f_1(x_1, x_2), f_2(x_1, x_2), f_3(x_1, x_2)\}$
  - **Recursion**: It is the process of repeating items in a self-similar way. It is related to mathematical induction. For e.g. Fibonacci sequence: $F_n = F_{n-1} + F_{n-2}$, initial condition $F_0=0$, $F_1= 1$, the factorial function: n! etc.
  - **μ –recursive function**: It is a partial function that can be constructed from the initial functions by a finite number of applications of the compositions, primitive recursions and minimization to regular functions.